UNIT-II

Machine to Machine (M2M) and Embedded Programming for IoT Machine-to-Machine Communications Overview, Difference between IoT and M2M, Interoperability in IoT: Standards and Protocols. Introduction to Arduino Programming for IoT, Integration of Sensors and Actuators with Arduino. Hands-on Exercises: Sensor Data Acquisition and Actuator Control, Basic Communication Protocols Implementation on Arduino, IoT Device Interoperability Challenges and Solutions.

2.1. Machine-to-Machine (M2M) Communications — Overview

M2M (Machine-to-Machine) refers to technologies that allow two or more devices (machines) to communicate and exchange data with each other **without human intervention**. It is the direct communication between devices using wired or wireless communication channels.

In short: M2M = Machine talks to Machine directly, mostly for monitoring and control.

2.1.1. M2M Architecture / Components

A typical M2M system has **three main building blocks**:

Component	Description
M2M Device	The end device (sensor, actuator, meter) that collects data or performs an action
M2M Area Network (Communication Network)	The network connecting devices to a gateway (e.g., ZigBee, Bluetooth, RFID, Wi-Fi)
M2M Gateway	Aggregates data from multiple devices and forwards it to the application server, often performing protocol conversion

Component	Description
Communication Network (WAN)	Wide area network (cellular, satellite, internet) connecting the gateway to the application/server
Application/Server	Processes the received data, stores it, and triggers actions or generates reports

Flow:

Sensor/Device → M2M Area Network → M2M Gateway → WAN (Cellular/Internet) → Application Server → End User

2.1.2. Key Characteristics of M2M

- Point-to-point communication (device to device or device to server)
- Uses telecom/cellular networks (2G/3G/4G/5G) or wired links, SCADA-based systems
- Data is typically sent to a central hub/server for processing
- Focused on **specific**, **vertical** (siloed) applications
- Uses embedded hardware modules and SIM-based communication frequently

2.1.3. M2M Application Areas

- Smart metering (electricity, water, gas)
- Fleet management and vehicle tracking (GPS)
- Industrial automation and remote monitoring (SCADA)
- Point-of-sale (POS) terminals
- Vending machine monitoring
- Healthcare remote patient monitoring
- ATM machine monitoring

2.1.4. Advantages of M2M

- Reduces need for manual monitoring and human error

- Enables real-time data collection and faster decision-making
- Increases operational efficiency and reduces costs
- Enables predictive maintenance

2.1.5. Challenges in M2M

- Lack of a common/open standard (many proprietary protocols)
- Limited scalability compared to IoT
- Security and privacy concerns
- High initial deployment cost for large-scale networks

2.2. Difference between IoT and M2M

Although M2M and IoT are related and often used interchangeably, they are **not the same**. IoT can be thought of as an evolution/superset of M2M.

2.1 Conceptual Difference

- **M2M**: Direct communication between machines, usually over telecom networks, in a **closed/siloed system**, primarily hardware-based.
- **IoT**: A larger ecosystem where devices connect to the **Internet**, use **IP-based** communication, and integrate with **cloud platforms, analytics, and applications** — creating an **open, interconnected system**.

2.2 Comparison Table

Basis	**M2M**	**IoT**
Communication	Point-to-point (device to device)	Uses internet, cloud, and network of networks
Connectivity	Hardware-based, often uses telecom/cellular modules	IP-based, uses Wi-Fi, Ethernet, cellular, IPv6, etc.

Basis	M2M	IoT
Scope	Limited/closed system, vertical (single application/silo)	Large-scale, open, horizontal (multiple applications)
Intelligence	Data mostly monitored, minimal analytics	Data analyzed using cloud/AI/ML for deep insights
Data Sharing	Data usually stays within its own system/vertical	Data can be shared across systems and applications
Human Involvement	Little to no human interaction	Human interacts via apps, dashboards, and interfaces
Protocols	SCADA, RS-232, proprietary telecom protocols	HTTP, MQTT, CoAP, XMPP, IPv6, 6LoWPAN, etc.
Scalability	Limited scalability	Highly scalable (billions of devices)
Standards	Mostly proprietary	Open standards (encourages interoperability)
Business Model	Hardware-driven, product-based	Service and data-driven (Platform as a Service, SaaS)
Example	Vending machine sending stock data to a supplier via SIM card	Smart home system where fridge, thermostat, and phone app share data via cloud

All M2M can be considered a subset of IoT, but not all IoT is M2M. M2M is a *technology enabler*, while IoT is a *complete ecosystem* built using M2M concepts plus internet connectivity, cloud computing, big data, and analytics.

2.3. Interoperability in IoT: Standards and Protocols

2.3.1 What is Interoperability?

Interoperability is the ability of different IoT devices, platforms, and systems — often built by different manufacturers using different technologies — to **connect, communicate, and exchange data meaningfully** with one another.

Since IoT involves a huge variety of devices (sensors, actuators, gateways) from different vendors, standard protocols are essential so that these heterogeneous devices can "understand" each other.

2.3.2 Types of Interoperability

Type	Meaning
Technical Interoperability	Devices/systems can physically connect and exchange bits (hardware/network layer)
Syntactic Interoperability	Data formats and structures are understood by both systems (e.g., JSON, XML)
Semantic Interoperability	The *meaning* of exchanged data is understood consistently by all systems
Organizational Interoperability	Different organizations/business processes can work together using IoT data

2.3.3. Why Standards Are Needed

- Ensures devices from different vendors can work together
- Avoids vendor lock-in
- Improves scalability and easier system integration
- Enhances security through well-tested, common protocols
- Reduces development cost and time

2.3.4. IoT Protocol Stack (by Layer)

(a) Link Layer Protocols

Responsible for physical/data transmission between nearby devices.

- **IEEE 802.15.4** – Low-rate wireless PAN standard, basis for ZigBee, 6LoWPAN
- **Wi-Fi (IEEE 802.11)** – High-speed wireless LAN
- **Bluetooth / BLE (Bluetooth Low Energy)** – Short-range, low-power communication
- **Z-Wave** – Home automation wireless protocol
- **RFID/NFC** – Identification and short-range communication

(b) Network/Internet Layer Protocols

Handles addressing and routing of packets.

- **IPv6** – Provides a huge address space needed for billions of IoT devices
- **6LoWPAN** (IPv6 over Low Power Wireless Personal Area Networks) – Compresses IPv6 headers for constrained devices
- **RPL** (Routing Protocol for Low Power and Lossy Networks) – Routing protocol for constrained mesh networks

(c) Transport Layer Protocols

- **UDP** – Lightweight, connectionless, used in constrained networks
- **TCP** – Reliable, connection-oriented (heavier for constrained devices)
- **DTLS** (Datagram Transport Layer Security) – Provides security over UDP

(d) Application Layer Protocols

These are the most important for data exchange between IoT applications:

Protocol	**Description**	**Key Features**
MQTT (Message Queuing Telemetry Transport)	Lightweight publish-subscribe protocol	Low bandwidth, ideal for constrained devices; uses a broker

Protocol	Description	Key Features
CoAP (Constrained Application Protocol)	Web transfer protocol for constrained nodes	REST-like, uses UDP, designed for M2M
HTTP/HTTPS	Standard web protocol	Widely used but heavier for constrained devices
XMPP (Extensible Messaging and Presence Protocol)	XML-based real-time communication	Supports presence detection, chat-like messaging
AMQP (Advanced Message Queuing Protocol)	Message-oriented middleware protocol	Reliable, used in enterprise-level messaging
DDS (Data Distribution Service)	Publish-subscribe protocol for real-time systems	Used in high-performance, real-time IoT systems

2.3.5 IoT Standardization Bodies/Organizations

- **IEEE** – Defines standards like 802.15.4, 802.11
- **IETF** (Internet Engineering Task Force) – Defines 6LoWPAN, RPL, CoAP
- **W3C** (World Wide Web Consortium) – Web of Things (WoT) standards
- **oneM2M** – Global standard specifically for M2M/IoT service layer interoperability
- **OASIS** – Standardizes MQTT
- **ETSI** – Telecommunications standards for M2M/IoT
- **ITU-T** – International Telecommunication standards for IoT

2.3.6 Approaches to Achieve Interoperability

1. **Standard Protocols** – Using common, open, well-defined protocols (as above)
2. **Middleware Platforms** – Software layer that translates/bridges different protocols (e.g., IoT gateways)
3. **Semantic Web Technologies** – Using ontologies (e.g., **oneM2M base ontology**, **SSN - Semantic Sensor Network**) to give shared meaning to data

4. **APIs (Application Programming Interfaces)** – RESTful APIs that allow different systems to exchange data in a standard way

2.4. IoT Device Interoperability — Challenges and Solutions

2.4.1.Why Interoperability Matters

IoT ecosystems consist of devices from **different vendors**, using **different hardware, protocols, and data formats**. Without interoperability, IoT systems become isolated "silos" that cannot share data or work together — limiting scalability and usefulness.

2.4.2. Key Challenges in IoT Interoperability

(a) Hardware Heterogeneity

- Devices use different microcontrollers, sensors, and communication modules (Wi-Fi, ZigBee, BLE, LoRa) with different capabilities (processing power, memory, energy).
- **Problem:** No single "one-size-fits-all" communication method works for all devices.

(b) Protocol Fragmentation

- Multiple competing protocols exist at every layer (MQTT vs CoAP vs HTTP vs AMQP; ZigBee vs Z-Wave vs Thread).
- **Problem:** Devices built on different protocol stacks cannot directly talk to each other.

(c) Data Format & Syntactic Incompatibility

- Different devices may send data in different formats (JSON, XML, binary, proprietary formats).
- **Problem:** Even if devices connect, the receiving system may not correctly parse the data.

(d) Semantic Incompatibility

- Even with the same data format, the **meaning** of the data can differ (e.g., temperature in Celsius vs Fahrenheit, different naming conventions for the same sensor type).
- **Problem:** Systems may misinterpret data, leading to wrong decisions/analytics.

(e) Security & Authentication Differences

- Different vendors implement different security mechanisms (encryption standards, authentication methods).
- **Problem:** Difficult to establish a uniform, secure trust model across heterogeneous devices.

(f) Vendor Lock-in / Proprietary Ecosystems

- Many manufacturers use proprietary protocols and closed ecosystems (e.g., closed cloud platforms) to retain customers.
- **Problem:** Limits integration with third-party devices/platforms, increasing cost and reducing flexibility.

(g) Scalability Issues

- As the number of connected devices grows (billions in IoT), maintaining interoperability across a large, dynamic, heterogeneous network becomes complex.

(h) Lack of Universal Standards

- No single globally accepted standard covers all IoT layers (unlike, say, TCP/IP for the traditional internet).
- **Problem:** Fragmented standardization efforts (oneM2M, IEEE, W3C, IETF) sometimes overlap or conflict.

2.4.3. Solutions to Interoperability Challenges

(a) Adoption of Open, Standard Protocols

- Using widely accepted, open application-layer protocols like **MQTT, CoAP, HTTP/HTTPS** instead of proprietary ones ensures devices can communicate using common "languages."

(b) Middleware & Gateway Solutions

- **IoT Gateways/Middleware** act as translators between different protocols (e.g., converting ZigBee data to MQTT/HTTP for cloud consumption).

- Enables legacy or proprietary devices to join a common network without hardware changes.

(c) Semantic Web Technologies & Ontologies

- Using shared **ontologies** (e.g., **SSN – Semantic Sensor Network**, **oneM2M base ontology**, **W3C Web of Things (WoT) Thing Description**) gives IoT data a common, machine-understandable meaning.
- Helps different systems interpret data (e.g., "temperature") the same way, solving semantic interoperability.

(d) Standardization Bodies & Frameworks

- Organizations like **oneM2M, IEEE, IETF, W3C, OCF (Open Connectivity Foundation)** work to create common service-layer standards so different vendors' devices can interoperate "out of the box."

(e) RESTful APIs and Web Standards

- Using **REST APIs** based on standard HTTP methods allows different platforms/applications to exchange data in a predictable, technology-agnostic way.

(f) Unified Data Formats

- Adopting common, lightweight data-interchange formats like **JSON** or **CBOR (Concise Binary Object Representation)** ensures data is easily parsed across systems, solving syntactic interoperability.

(g) Edge Computing & Protocol Translation at the Edge

- Performing protocol translation and data normalization closer to the device (at the edge/gateway) reduces the burden on the cloud and allows heterogeneous devices to present a uniform interface to the rest of the system.

(h) Security Standardization

- Adopting standard security protocols like **TLS/DTLS**, **OAuth 2.0**, and standardized device identity/authentication frameworks helps build a consistent trust model across vendors.

(i) Open Alliances / Industry Consortiums

- Groups like the **Open Connectivity Foundation (OCF)**, **Thread Group**, and **Matter (by CSA – Connectivity Standards Alliance)** are industry-driven efforts specifically to solve smart-home/IoT interoperability by defining a common application-layer standard across vendors.
    - **Matter** (built on IP, Thread, and Wi-Fi) is a notable modern example enabling smart home devices from different brands (Google, Amazon, Apple, Samsung, etc.) to work together.

2.5. Introduction to Arduino Programming for IoT

Arduino is an open-source electronics platform based on easy-to-use hardware and software, widely used in IoT projects for prototyping and embedded system development.

2.5.1.Arduino Basics and Setup

Arduino IDE and Program Structure

The Arduino IDE (Integrated Development Environment) is where you write, compile, and upload code to your Arduino board. Every Arduino program (called a "sketch") has two main functions:

- **setup()**: Runs once when the Arduino starts. Used to initialize pins, start serial communication, and configure sensors.
- **loop()**: Runs repeatedly after setup(). This is where your main logic executes continuously.

Example basic structure: cpp

```
void setup() {
  Serial.begin(9600);     // Start serial communication at 9600 baud
  pinMode(13, OUTPUT);    // Set pin 13 as output (built-in LED)
  pinMode(A0, INPUT);     // Set pin A0 as input (for analog sensor)
}
```

```
void loop() {
  int sensorValue = analogRead(A0);  // Read analog sensor
  Serial.println(sensorValue);      // Print to Serial Monitor
  digitalWrite(13, HIGH);          // Turn on LED
  delay(1000);                    // Wait 1 second
  digitalWrite(13, LOW);           // Turn off LED
  delay(1000);
}
```

Key Programming Concepts

- **pinMode(pin, mode)**: Configures a pin as INPUT, OUTPUT, or INPUT_PULLUP.
- **digitalRead(pin)** / **digitalWrite(pin, value)**: Reads or writes digital (HIGH/LOW) signals.
- **analogRead(pin)**: Reads analog values (0-1023 for 10-bit ADC on Arduino Uno).
- **Serial.begin(baud)** / **Serial.println(data)**: Enables serial communication for debugging.

2.5.2.Integration of Sensors and Actuators with Arduino

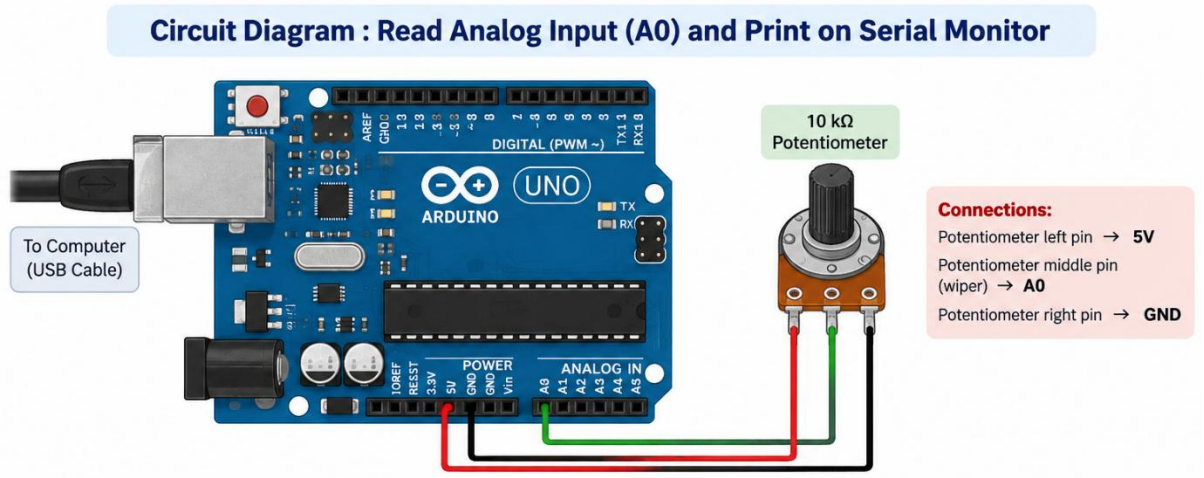
Sensors: The "Eyes and Ears" of IoT

Sensors convert physical quantities (temperature, light, distance, etc.) into electrical signals that Arduino can read. Common sensor types include:

- **Digital Sensors**: Output HIGH/LOW signals (e.g., push buttons, PIR motion sensors).
- **Analog Sensors**: Output variable voltage (e.g., LDR for light, potentiometer, temperature sensors like LM35).
- **Communication-based Sensors**: Use protocols like I2C or SPI (e.g., MPU6050 accelerometer, OLED displays).

Example: Reading a Potentiometer (Analog Sensor)

Circuit diagram:



Working:

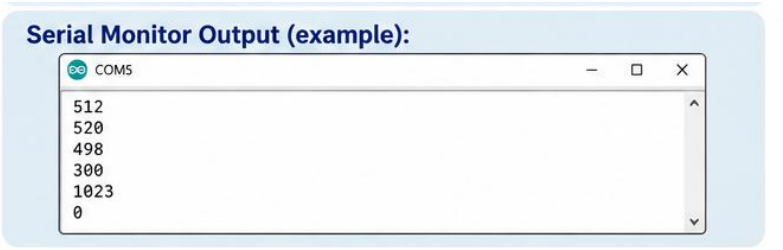
- The potentiometer provides a variable voltage (0–5V) to analog pin **A0**.
- Arduino reads the analog value (0–1023) using `analogRead(A0)`.
- The value is printed on the Serial Monitor every 500 ms.

Program:

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  int value = analogRead(A0);      // Read analog pin A0 (0-1023)
  Serial.println(value);           // Print value to Serial Monitor
  delay(500);
}
```

Output:



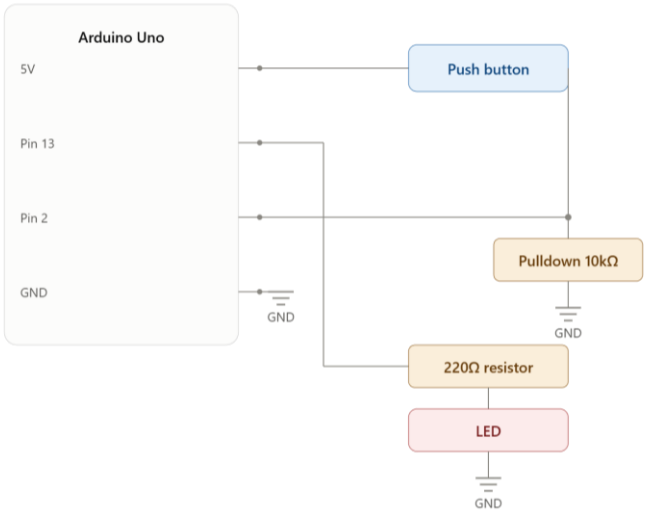
Actuators: The "Hands" of IoT

Actuators convert electrical signals from Arduino into physical action (movement, light, sound, etc.). Common actuators include:

- **LEDs**: Simple digital output (HIGH = ON, LOW = OFF).
- **Relays**: Control high-voltage devices (fans, lamps) using low-voltage Arduino signals.
- **Servo Motors**: Precise angular control (0-180°).
- **DC Motors**: Continuous rotation (requires motor driver like L298N).
- **Buzzers**: Generate sound.

Example: Controlling an LED with a Button

Circuit Diagram:



Program Code:

```
const int buttonPin = 2;

const int ledPin = 13;


void setup() {
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);
}


void loop() {
  int buttonState = digitalRead(buttonPin);
  if (buttonState == HIGH) {
    digitalWrite(ledPin, HIGH);    // Turn LED ON
  } else {
    digitalWrite(ledPin, LOW);     // Turn LED OFF
  }
}
```

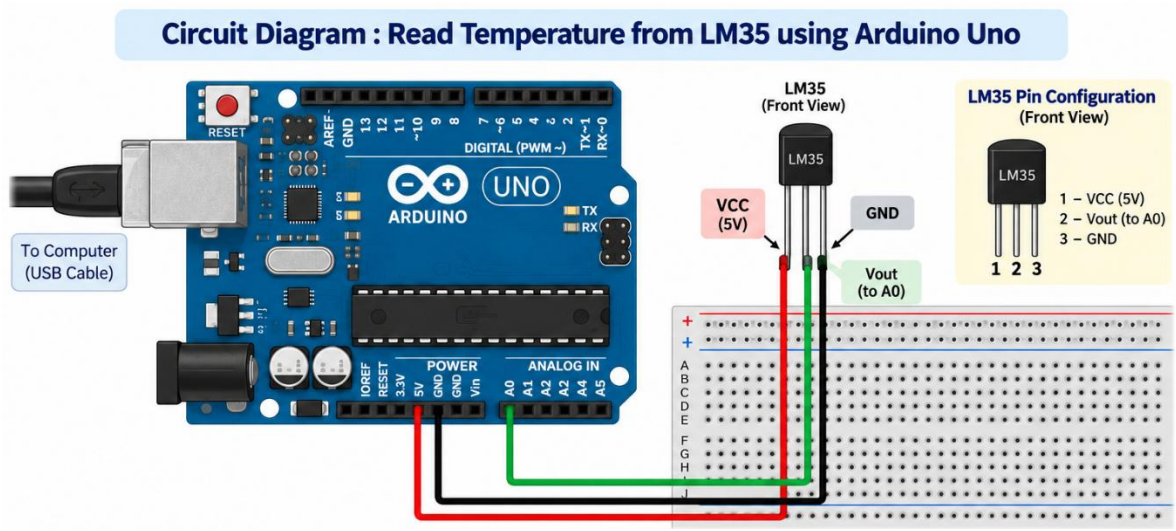
2.6.Hands-on Exercises: Sensor Data Acquisition and Actuator Control

2.6.1. Exercise 1: Temperature Monitoring System

Objective: Read temperature from an LM35 sensor and display it on Serial Monitor.

Components: Arduino Uno, LM35 temperature sensor, breadboard, jumper wires.

Circuit Diagram:



- LM35 VCC → Arduino 5V
- LM35 GND → Arduino GND
- LM35 Output → Arduino A0

Working:

- LM35 provides 10 mV per °C.
- Arduino reads the analog value (0–1023).
- Value is converted to voltage (0–5V).
- Temperature (°C) = Voltage × 100.
- The temperature is printed on the Serial Monitor every 1 second.

Program Code:

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  int sensorValue = analogRead(A0);          // Read analog value (0-1023)
```

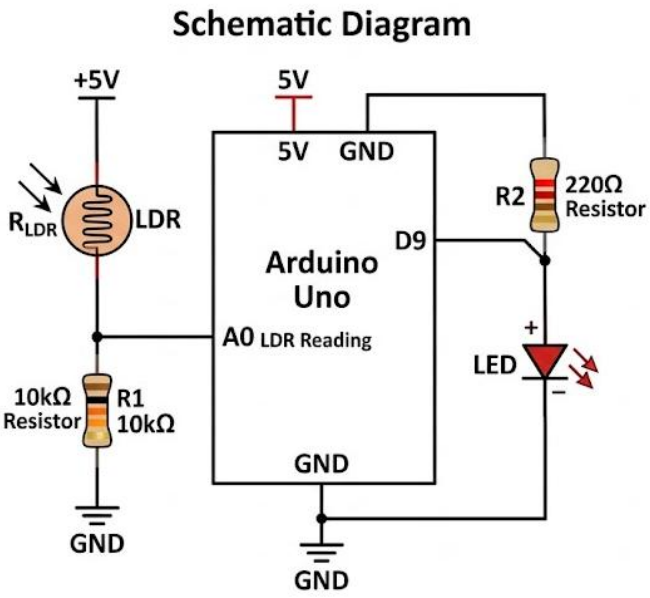
```
  float voltage = sensorValue * (5.0 / 1023); // Convert to voltage (0-5V)
  float temperature = voltage * 100;          // LM35: 10mV per °C
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" °C");
  delay(1000);
}
```

2.6.2.Exercise 2: Automatic Light Control System

Objective: Use an LDR (Light Dependent Resistor) to detect ambient light and control an LED automatically.

Components: Arduino Uno, LDR, 10kΩ resistor, LED, 220Ω resistor, breadboard.

Circuit Diagram:



- LDR + 10kΩ resistor form a voltage divider connected to A0.
- LED (+) → 220Ω resistor → Arduino pin 9.
- LED (-) → Arduino GND.

Program Code:

```
const int ldrPin = A0;

const int ledPin = 9;

const int threshold = 500;  // Adjust based on testing


void setup() {

  pinMode(ledPin, OUTPUT);

  Serial.begin(9600);

}


void loop() {

  int lightLevel = analogRead(ldrPin);

  Serial.println(lightLevel);


  if (lightLevel < threshold) {  // Dark environment

    digitalWrite(ledPin, HIGH);  // Turn LED ON

  } else {

    digitalWrite(ledPin, LOW);   // Turn LED OFF

  }

  delay(200);

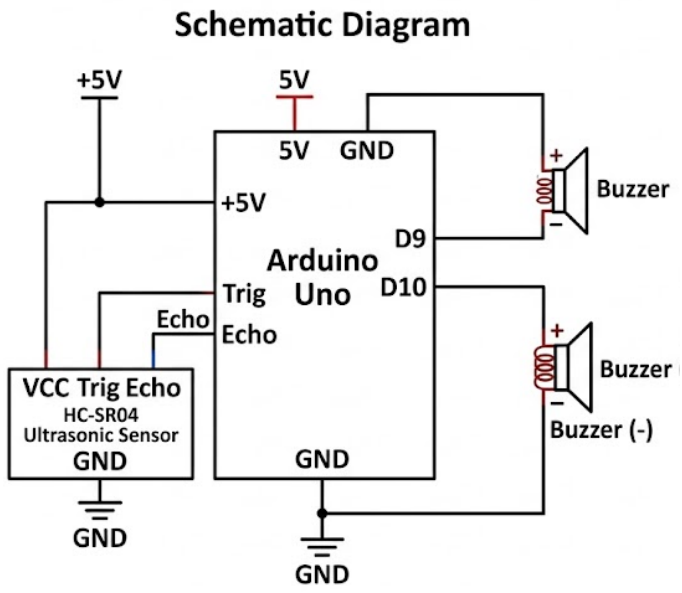
}
```

2.6.3.Exercise 3: Ultrasonic Distance Measurement with Buzzer Alert

Objective: Measure distance using HC-SR04 ultrasonic sensor and trigger a buzzer when an object is too close.

Components: Arduino Uno, HC-SR04 ultrasonic sensor, buzzer, breadboard.

Circuit Diagram:



- HC-SR04 VCC → 5V, GND → GND
- Trig → Pin 9, Echo → Pin 10
- Buzzer (+) → Pin 11, Buzzer (-) → GND

Program Code:

```
const int trigPin = 9;

const int echoPin = 10;

const int buzzerPin = 11;



void setup() {

  pinMode(trigPin, OUTPUT);

  pinMode(echoPin, INPUT);

  pinMode(buzzerPin, OUTPUT);

  Serial.begin(9600);
```

```
}

void loop() {
  // Send 10µs pulse to Trig
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Read Echo pulse duration
  long duration = pulseIn(echoPin, HIGH);

  // Calculate distance (speed of sound = 343 m/s)
  float distance = duration * 0.0343 / 2;  // in cm

  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");

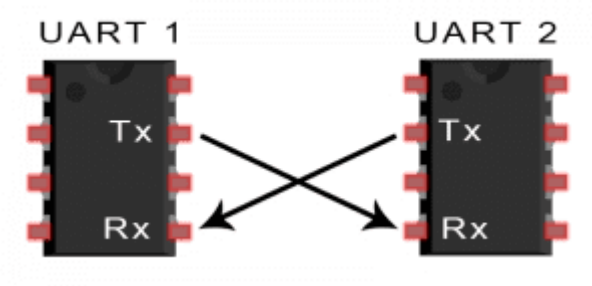
  if (distance < 10) {  // Object closer than 10 cm
    digitalWrite(buzzerPin, HIGH);
  } else {
```

```
  digitalWrite(buzzerPin, LOW);
 }
 delay(200);
}
```

2.7.Basic Communication Protocols Implementation on Arduino

Arduino supports three primary serial communication protocols for interfacing with sensors, displays, and other modules: UART, I2C, and SPI.

2.7.1. UART (Universal Asynchronous Receiver-Transmitter)



Characteristics:

- Asynchronous (no clock signal), uses TX (transmit) and RX (receive) pins.
- Point-to-point communication (one-to-one).
- Requires common ground between devices.
- Speed defined by baud rate (e.g., 9600, 115200).

Arduino Pins (Uno): TX = Pin 1, RX = Pin 0 (also used for USB serial).

Use Cases: GPS modules, Bluetooth (HC-05), serial communication between two Arduinos.

Example: UART Communication with Serial Monitor

Circuit Diagram:

Program Code:

```
void setup() {
  Serial.begin(9600);  // Initialize UART at 9600 baud
}


void loop() {
  Serial.println("Hello from Arduino!");
  delay(1000);
}
```

Example: UART Between Two Arduinos

- Connect Arduino 1 TX → Arduino 2 RX
- Connect Arduino 1 RX → Arduino 2 TX
- Connect GND → GND

2.7.2.I2C (Inter-Integrated Circuit)

Characteristics:

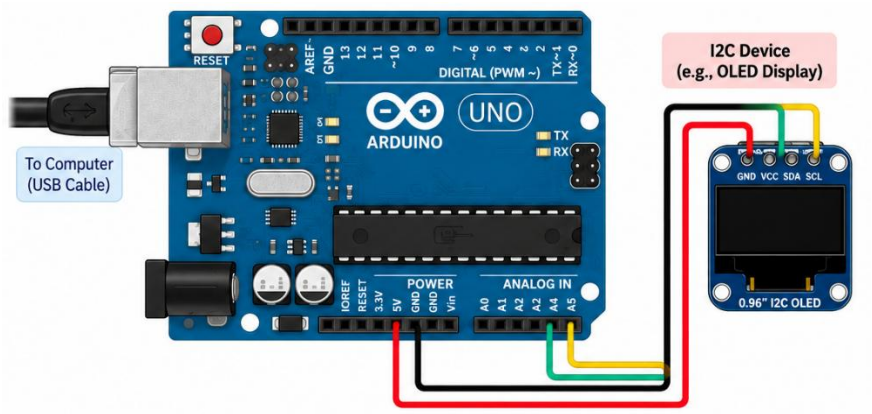
- Synchronous (uses clock signal), 2-wire interface: SDA (data) and SCL (clock).
- Supports multiple devices on the same bus (each has a unique 7-bit or 10-bit address).
- Requires pull-up resistors (typically 4.7kΩ) on SDA and SCL lines.

Arduino Pins (Uno): SDA = A4, SCL = A5.

Use Cases: OLED displays, MPU6050 accelerometer, RTC modules, multiple sensors on same bus.

Example: I2C Scanner (Find Connected Devices)

Circuit Diagram:



Connections:

I2C Device **VCC**	→	Arduino **5V**
I2C Device GND	→	Arduino **GND**
I2C Device SDA	→	Arduino **A4** (SDA)
I2C Device SCL	→	Arduino **A5** (SCL)

Note:

- I2C uses only two data lines: SDA (data) and SCL (clock).
- Multiple I2C devices can be connected to the same A4 and A5 lines.
- Here an OLED display is shown as an example. You can connect any I2C device (e.g., RTC, sensor, EEPROM, etc.).

Program Code:

```
#include <Wire.h>


void setup() {
  Wire.begin();
  Serial.begin(9600);
  Serial.println("I2C Scanner");
}


void loop() {
  int count = 0;
```

```
  for (byte addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) {
      Serial.print("Device found at address: 0x");
      Serial.println(addr, HEX);
      count++;
    }
  }
  if (count == 0) {
    Serial.println("No devices found!");
  }
  delay(5000);
}
```

Example Serial Monitor Output:



Example: Writing to I2C Device (e.g., OLED at 0x3C)-cpp

```
#include <Wire.h>

void setup() {
  Wire.begin();
  Wire.beginTransmission(0x3C);  // I2C address of OLED
```

```
  Wire.write(0x00);          // Command byte
  Wire.endTransmission();
}

void loop() {
  // Main code
}
```

2.7.3.SPI (Serial Peripheral Interface)

Characteristics:

- Synchronous, full-duplex, high-speed communication.
- Uses 4 wires: MOSI (Master Out Slave In), MISO (Master In Slave Out), SCK (clock), SS/CS (slave select).
- One master can control multiple slaves (each needs separate SS pin).

Arduino Pins (Uno):

- MOSI = Pin 11
- MISO = Pin 12
- SCK = Pin 13
- SS = Pin 10 (or any digital pin)

Use Cases: SD cards, TFT displays, high-speed sensors, RFID modules (RC522).

Example: SPI Communication Setup

Program Code:

```
#include <SPI.h>

const int slaveSelectPin = 10;
```

```
void setup() {
  pinMode(slaveSelectPin, OUTPUT);
  SPI.begin();
  Serial.begin(9600);
}

void loop() {
  digitalWrite(slaveSelectPin, LOW);  // Select slave device
  SPI.transfer(0xAA);                 // Send data
  digitalWrite(slaveSelectPin, HIGH); // Deselect slave
  delay(1000);
}
```

2.8.Protocol Comparison Table

Feature	**UART**	**I2C**	**SPI**
Wires	2 (TX, RX) + GND	2 (SDA, SCL) + GND	4+ (MOSI, MISO, SCK, SS) + GND
Speed	Medium (up to ~1 Mbps)	Medium (up to ~400 kHz standard, 3.4 MHz high-speed)	High (up to ~10+ Mbps)
Devices	Point-to-point (1-to-1)	Multi-device (1-to-many, addressed)	Multi-device (1-to-many, separate SS per device)
Complexity	Simple	Moderate (addressing, pull-ups)	Moderate (more wires, SS management)

Feature	UART	I2C	SPI
Typical Use	GPS, Bluetooth, serial debug	Sensors, displays, RTC	SD cards, displays, high-speed modules